

Alias Resolution Engine: Working and Design

1. Purpose of Aliasing Engine

The Alias Resolution Engine is a **review-driven system** that normalizes noisy transaction narratives to canonical counterparties (customers/vendors) with **strong correctness guarantees**.

It prioritizes deterministic resolution, explicit approval for ambiguous cases, permanent learning of approved aliases, and fast reuse of past decisions.

It is **not** an auto-fuzzy or ML-driven resolver. Fuzzy matching is used **only to assist ranking**, not to override approved mappings.

2. High-Level Resolution Flow

```
Python
Raw transaction narrative
    ↓
Parsed using CTPTY function into counterparties(payer & payee)
    ↓
Exact alias lookup
    ↓
If found → return canonical counterparty
    ↓
Else → generate ranked fuzzy suggestions
    ↓
Approval decision from user
    ↓
Persist alias mapping to alias table
```

3. Core Resolution Rules

Alias Resolution Engine Principles:

- **Exact match is absolute:** If an alias exists, fuzzy logic is bypassed.
 - **Fuzzy matching is assistive only:** Used for ranking suggestions, never for auto-commitment.
 - **Approved mappings are permanent:** Stored aliases are reused for all future transactions.
 - **No unresolved telemetry:** Skipped or rejected aliases are not persisted.
-

4. Current Persistence Model (Present State)

`aliases.json` — Alias Registry (Current)

Role : Acts as the authoritative registry of approved alias mappings.

What it stores

- Normalized alias text
- Associated `ctpty_id` (counterparty UUID) and canonical name approved
- Approval metadata

Python

```
{  
  
    "ctpty_id": "b6d5d8b5-...",  
  
    "alias_raw": "ac 1234567890 abc trading",  
  
    "alias_norm": "AC-1234567890 ABC TRADING",  
  
    "approved_by": "sidd",  
  
    "approved_at": "2025-12-29T12:00:00Z"  
  
    "canonical_name": "ABC Trading Pvt Ltd",  
  
}
```

5. Planned Persistence Model (Future State)

★ Master Alias Table (PostgreSQL)

The system is designed to transition from file-based storage to a **database alias registry**.

Future design changes

- `aliases.json` will be replaced by a **PostgreSQL alias table**
- The alias table will store:
 - `alias_norm`
 - `ctpty_id`
 - approval metadata
- **No canonical name stored in alias table**

Canonical name resolution will always occur via:

None

`alias → ctpty_id → counterparty master table`

This ensures:

- Single source of truth for canonical names
- No duplication of names across systems
- Safer updates if counterparty names change

Conceptual schema:

SQL

```
alias_master (  
    alias_norm : TEXT PRIMARY KEY,  
    ctpty_id : UUID NOT NULL,  
    approved_by : TEXT,  
    approved_at : TIMESTAMP
```

)

6. Global Master Cache (Read-Optimized)

6.1 Purpose

The engine optionally uses a **global in-memory master list** of counterparties across all tenants to improve fuzzy suggestion quality and eliminate repeated DB queries.

6.2 Behavior

- Loaded once per process
- Stores:
 - `ctpty_id`
 - canonical name
 - normalized name
- Used **only for suggestion ranking**
- Does **not** define correctness

6.3 Manual Injection : When a new counterparty is approved:

- It is injected into the in-memory master cache
 - Immediately becomes available for fuzzy ranking
 - No restart required
-

7. Resolution Engine (`resolve.py`)

Primary responsibilities

- Normalize raw text

- Perform exact alias lookup
 - Generate ranked fuzzy suggestions
 - Capture approval decisions
 - Persist approved mappings
 - Update master cache for immediate reuse
-

8. Fuzzy Matching Strategy

Fuzzy matching is **ranking-only**, never authoritative.

Scoring components

- RapidFuzz token-set ratio
- RapidFuzz token-sort ratio
- Jaro-Winkler similarity

Python

```
ts = fuzz.token_set_ratio(a, b) / 100 # token overlap
tr = fuzz.token_sort_ratio(a, b) / 100 # token order / positioning
jw = jellyfish.jaro_winkler_similarity(a, b) # prefix similarity

score = (0.25 * ts) + (0.25 * tr) + (0.50 * jw)
```

Output : Top-K ranked suggestions